PANDAS WITH PYTHON HANDBOOK

PAGE 1 : PANDAS FUNDAMENTALS

1. What is Pandas?

- Pandas is a powerful open-source library in Python used for data manipulation and analysis.
- It provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive.
- Pandas is built on top of NumPy.

2. Why Pandas is used with Python?

- Easy data handling and manipulation.
- High-performance data analysis tools.
- Powerful functions for cleaning, transforming and analyzing data.
- Works well with many data sources and formats.
- Widely used in Data Science, Machine Learning and Analytics.

3. Installing & Importing Pandas

```
# Install Pandas
pip install pandas
```

```
# Import Pandas
import pandas as pd
```

4. Series vs DataFrame

- Series : One-dimensional labeled array (like a single column).
  DataFrame : Two-dimensional labeled data structure (like a table).

5. Creating a Series

```
import pandas as pd
data = [10, 20, 30, 40]
s = pd.Series(data, name='Numbers')
print(s)
```

Create a Series using a list

6. Creating a DataFrame

```
import pandas as pd
data = {'Name': ['Aman', 'Rohit', 'Neha'],
        'Age':  [22, 24, 21],
        'City': ['Delhi', 'Mumbai', 'Pune']}
df = pd.DataFrame(data)
print(df)
```

Create a DataFrame using a dictionary

7. Reading Basic Data

- pd.read_csv('file.csv') # Read CSV file
- pd.read_excel('file.xlsx') # Read Excel file
- pd.read_json('file.json') # Read JSON file

8. Basic DataFrame Inspection

- df.head() # First 5 rows
- df.tail() # Last 5 rows
- df.shape # (rows, columns)
- df.columns # Column names
- df.dtypes # Data types of columns
- df.info() # Summary of DataFrame
- df.describe() # Statistical summary (numeric cols)

PANDAS WITH PYTHON HANDBOOK

PAGE 2 : SELECTING & FILTERING DATA

1. Selecting Columns

- To select a single column, use the column name.

```
df['Name']                    # Returns a Series
```

2. Selecting Multiple Columns

- Select two or more columns using a list of column names.

```
df[['Name', 'Age', 'City']]   # Returns a DataFrame
```

3. Selecting Rows

- Select rows by their index label using loc[] or by position using iloc[].

```
df.loc[0]                     # Row with index label 0 (Series)
df.iloc[0]                    # Row at position 0 (Series)
```

4. loc[] vs iloc[]

loc[] (Label-based)	iloc[] (Integer Position-based)
• Uses row/column labels.	• Uses integer positions.
• Includes the last label in a range.	• Does NOT include the last index in a range.
• Can use boolean conditions.	• Only integers allowed.

5. Conditional Filtering

- Filter rows based on a single condition.

```
df[df['Age'] > 25]            # Returns rows where Age is greater than 25
```

6. Multiple Conditions

- Use & (and), | (or) and parentheses.

```
df[(df['Age'] > 25) & (df['City'] == 'Delhi')]      # Multiple conditions
```

7. Using isin()

- Filter rows where a column value is in a list of values.

```
df[df['City'].isin(['Delhi', 'Mumbai'])]      # City is either Delhi or Mumbai
```

8. Using between()

- Filter rows between a range of values.

```
df[df['Age'].between(20, 30)]         # Age between 20 and 30 (inclusive)
```

9. Using query()

- Use query() for readable and efficient filtering.

```
df.query("Age > 25 and City == 'Pune'")      # Query using column names as string
```

10. Boolean Indexing

- Create a condition and use it to filter DataFrame.

```
condition = df['Age'] >= 30
df[condition]                 # Filter using a boolean condition
```

PANDAS WITH PYTHON HANDBOOK

PAGE 3 : HANDLING MISSING & DUPLICATE DATA

1. Detecting Missing Values

- Pandas represents missing values as NaN (Not a Number).
- Common reasons: incomplete data entry, unavailable values, errors in collection.

2. Checking for Missing Values

- isna() / isnull() → Returns True for missing values.

```
df.isna()    # or df.isnull()
```

- notna() → Returns True for non-missing values.

```
df.notna()
```

- Count total missing values in each column.

```
df.isna().sum()
```

How isna() works?

Original DataFrame

Name	Age	City
Aman	22	Delhi
Neha	NaN	Mumbai
Rohit	25	NaN
Priya	NaN	Pune

isna() →

After isna()

Name	Age	City
False	False	False
False	True	False
False	False	True
False	True	False

3. Dropping Missing Values

- dropna() → Removes rows/columns with missing values.

```
df.dropna()              # Drop rows with any NaN
df.dropna(axis=1)        # Drop columns with any NaN
```

- Drop rows only if all values are NaN.

```
df.dropna(how='all')
```

- Drop rows where specific subset has NaN.

```
df.dropna(subset=['Age','City'])
```

4. Filling Missing Values

- fillna() → Replaces NaN with a value.

```
df.fillna(0)                    # Fill with 0
df.fillna('Unknown')            # Fill with string
df.fillna(df['Age'].mean())     # Fill with column mean
df.fillna(method='ffill')       # Forward fill (propagate last valid value)
df.fillna(method='bfill')       # Backward fill (propagate next valid value)
```

5. Replacing Invalid or Incorrect Values

- Replace specific values with another value.

```
df.replace('N/A', np.nan)
df.replace({'Male': 'M', 'Female': 'F'})      # Replace using dictionary
```

6. Handling Duplicates

- Detect duplicated rows.

```
df.duplicated()                 # Returns True for duplicate rows
df.duplicated(keep='first')     # Mark duplicates except first occurrence
```

- Remove duplicate rows.

```
df.drop_duplicates()            # Remove all duplicates (keep first)
df.drop_duplicates(keep='last') # Keep last occurrence
```

7. Summary of Common Functions

Function	Purpose	Example
isna() / isnull()	Check missing values (NaN)	df.isna()
notna()	Check non-missing values	df.notna()
dropna()	Remove rows/columns with NaN	df.dropna()
fillna()	Fill NaN with value or method	df.fillna(0)
replace()	Replace old values with new values	df.replace('N/A', np.nan)
duplicated()	Identify duplicate rows	df.duplicated()
drop_duplicates()	Remove duplicate rows	df.drop_duplicates()

PANDAS WITH PYTHON HANDBOOK

PAGE 4 : DATA TRANSFORMATION

1. Creating New Columns

- You can create a new column by assigning values or expressions.

```
df['Total'] = df['Quantity'] * df['Price']
df['Discounted_Price'] = df['Price'] * 0.9
```

Create columns using existing columns and operations

2. Updating Existing Columns

- You can update the values of an existing column.

```
df['Price'] = df['Price'] + 10
df.loc[df['Stock'] < 10, 'Status'] = 'Low Stock'
```

Update column values using conditions

3. Using map()

- Used to replace values of a column based on a mapping.

```
mapping = {'M':'Male', 'F':'Female'}
df['Gender'] = df['Gender'].map(mapping)
```

Map values using a dictionary

4. Using apply()

- Applies a function along an axis (rows or columns).

```
df['Name_Length'] = df['Name'].apply(len)
df['Tax'] = df['Amount'].apply(lambda x: x * 0.18)
```

Apply a function element-wise

5. Using replace()

- Replace old values with new values.

```
df.replace({'Yes': 1, 'No': 0}, inplace=True)
df.replace({'N/A': None}, inplace=True)
```

Replace values in DataFrame

6. Using assign()

- Create new columns and assign them in a single step.

```
df = df.assign(
   Profit = df['Revenue'] - df['Cost'],
   Margin = lambda x: (x['Profit'] / x['Revenue']) * 100
)
```

Assign multiple columns together

7. Using Lambda Functions

- Lambda functions are often used with apply() and assign().

```
df['Category'] = df['Amount'].apply(lambda x: 'High' if x > 1000 else 'Low')
```

Lambda with apply()

8. Changing Data Types

- Convert column data type using astype().

```
df['Quantity'] = df['Quantity'].astype(int)
df['Price'] = df['Price'].astype(float)
df['Date'] = df['Date'].astype('datetime64[ns]')
```

9. Renaming Columns

- Rename column(s) for better readability.

```
df.rename(columns={'OldName':'NewName'},
                  inplace=True)
# Or
df.rename(columns=str.lower, inplace=True)
```

10. set_index() and reset_index()

set_index()

- Sets a column as index.
- Useful for time series or unique identifiers.

```
df.set_index('Date', inplace=True)
```

reset_index()

- Resets the index.
- Converts index back to a column.

```
df.reset_index(inplace=True)
```

PANDAS WITH PYTHON HANDBOOK

PAGE 5 : SORTING & AGGREGATION

1. Sorting Rows with sort_values()

- Sort rows by the values of one or more columns.

```
df.sort_values('Age')                          # Ascending (default)
df.sort_values('Age', ascending=False)         # Descending
df.sort_values(['City','Age'])                 # Sort by multiple columns
df.sort_values(['City','Age'], ascending=[True, False])
```

2. Sorting by Index with sort_index()

- Sort rows by index labels.

```
df.sort_index()                    # Ascending
df.sort_index(ascending=False)     # Descending
```

3. Sorting by Multiple Columns

- You can sort by multiple columns with different order.

```
df.sort_values(by=['City','Age','Name'],
               ascending=[True, False, True])
```

4. value_counts()

- Returns counts of unique values in a Series.

```
df['City'].value_counts()
df['City'].value_counts(normalize=True)    # Proportions
```

5. unique() and nunique()

- unique() → Returns array of unique values.
- nunique() → Returns count of unique values.

```
df['City'].unique()      # Unique values
df['City'].nunique()     # Count of unique values
```

6. Aggregation with groupby()

- Group data by one column and apply aggregation.

```
df.groupby('City')['Age'].sum()
df.groupby('City')['Age'].mean()
df.groupby('City')['Age'].count()
df.groupby('City')['Age'].min()
df.groupby('City')['Age'].max()
```

7. Grouping by Multiple Columns

- Group by two or more columns.

```
df.groupby(['City','Gender'])['Salary'].mean()
df.groupby(['City','Gender']).size()    # Count
```

8. Using agg() for Multiple Aggregations

- Apply different aggregation functions.

```
df.groupby('City')['Salary'].agg(['sum','mean','max','min','count'])
```

9. Named Aggregations

- Create custom names for aggregated columns.

```
df.groupby('City').agg(
    total_salary=('Salary','sum'),
    avg_salary=('Salary','mean'),
    emp_count=('Salary','count')
)
```

10. Real-world Example

- Example: Sales Data Analysis

Sample DataFrame: sales

Date	Region	Product	Sales	Quantity
2024-01-01	North	Laptop	60000	2
2024-01-01	North	Mouse	1000	5
2024-01-02	South	Laptop	55000	1
2024-01-02	South	Keyboard	2000	3
2024-01-03	North	Keyboard	2000	4
2024-01-03	South	Mouse	800	2

- Some Useful Aggregations
  - Total sales by region

```
sales.groupby('Region')['Sales'].sum()
```

  - Total quantity sold by product

```
sales.groupby('Product')['Quantity'].sum()
```

  - Average sales by region and product

```
sales.groupby(['Region','Product'])['Sales'].mean()
```

  - Maximum sales in each region

```
sales.groupby('Region')['Sales'].max()
```

11. transform() vs agg()

agg()

- Reduces the data (returns aggregated result).
- Output has one row per group.
- Used for summary statistics.

Example:

```
df.groupby('City')['Sales'].sum()
```

transform()

- Returns a result with the same length as original.
- Broadcasts the aggregated result back to original rows.
- Useful for grouped calculations on every row.

Example:

```
df['Total_Sales_City'] = df.groupby('City')['Sales'].transform('sum')
```

PANDAS WITH PYTHON HANDBOOK

PAGE 6 : COMBINING DATAFRAMES

1. concat() : Concatenate DataFrames

- Used to combine DataFrames along rows or columns.
- By default, it concatenates along rows (axis=0).

```
df1 = pd.DataFrame({'A':[1,2], 'B':[3,4]})
df2 = pd.DataFrame({'A':[5,6], 'B':[7,8]})
result = pd.concat([df1, df2])            # axis=0 (rows)
result = pd.concat([df1, df2], axis=1)    # columns
```

Key Parameters:

- axis=0 → Row-wise (default)
- axis=1 → Column-wise
- ignore_index=True → Reindex the result
- join='outer'/'inner' → Handle non-matching columns

Example (Row-wise)

df1

Index	A	B
0	1	3
1	2	4

df2

Index	A	B
0	5	7
1	6	8

↓

Result (concat axis=0)

Index	A	B
0	1	3
1	2	4
0	5	7
1	6	8

2. merge() : Database-style Join

- Used to merge/join DataFrames similar to SQL joins.
- Requires a key column (or columns) to join on.

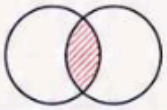
```
pd.merge(left_df, right_df, on='key', how='inner')
```

Common how options:

- 'inner' → Only matching rows (intersection)
- 'left' → All rows from left, matched from right
- 'right' → All rows from right, matched from left
- 'outer' → All rows from both (union)

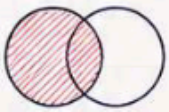
3. Types of Joins in merge()

- Inner Join (how='inner')



Returns only the matching records.

- Left Join (how='left')



All records from left + matching from right.

- Right Join (how='right')



All records from right + matching from left.

- Outer Join (how='outer')



All records from both DataFrames.

4. join() : Join on Index

- Join is a convenient method for index-based merging.
- Similar to merge(), but works on index by default.

```
df1.join(df2, how='left')    # Join on index
df1.join(df2, how='inner')
```

5. merge() vs join()

Feature	merge()	join()
Basis	Column(s)	Index
Syntax	pd.merge(df1, df2, on='key')	df1.join(df2)
Flexibility	More flexible	Less flexible
Use Case	SQL-style joins	Index-based joins

6. Handling Duplicate Column Names

- While merging, if both DataFrames have same column names (except key), Pandas adds suffixes.

```
pd.merge(df1, df2, on='key', how='inner', suffixes=('_left', '_right'))    # default: _x, _y
```

7. Summary : When to Use What?

Function	Use When
concat()	When you want to stack DataFrames (vertically or horizontally).
merge()	When you want to join based on column(s) (like SQL joins).
join()	When you want to join based on index.

PANDAS WITH PYTHON HANDBOOK

PAGE 7 : STRING & DATETIME OPERATIONS

1. String Operations with .str accessor

- Used to perform vectorized string operations on Series.

```
s = pd.Series(['Apple', 'banana', 'Cherry', None])
s.str.method()              # Apply string methods
```

2. Common String Methods

Method	Description	Example
str.lower()	Convert to lowercase	s.str.lower()
str.upper()	Convert to uppercase	s.str.upper()
str.title()	Convert to title case	s.str.title()
str.strip()	Remove leading/trailing spaces	s.str.strip()
str.replace(old, new)	Replace substring	s.str.replace('a','x')
str.contains(pat)	Check if pattern exists	s.str.contains('app')
str.startswith(pat)	Check if starts with	s.str.startswith('A')
str.endswith(pat)	Check if ends with	s.str.endswith('y')
str.len()	Length of each string	s.str.len()
str.split(sep)	Split strings by separator	s.str.split(',')

3. Extracting Text

- Extract parts of the string using extract() with regex.

```
s = pd.Series(['ID:101-A', 'ID:102-B', 'ID:103-C'])
s.str.extract(r'ID:(\d+)-([A-Z])')
                            # Extract number and letter
```

4. String Example

- Clean and transform string data.

```
df['Name'] = df['Name'].str.strip().str.title()
df['Email_Domain'] = df['Email'].str.split('@').str[1]
df['Has_Apple'] = df['Name'].str.contains('Apple', case=False)
```

5. Converting to Datetime

- Convert string/object column to datetime.

```
df['Date'] = pd.to_datetime(df['Date'])
# You can specify format if known
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')
```

6. Extracting Date Parts

- Access parts of datetime using .dt accessor.

```
df['Year']      = df['Date'].dt.year
df['Month']     = df['Date'].dt.month
df['Day']       = df['Date'].dt.day
df['Weekday']   = df['Date'].dt.day_name()
df['DayOfWeek'] = df['Date'].dt.dayofweek  # Mon=0, Sun=6
df['Quarter']   = df['Date'].dt.quarter
```

7. Date Filtering

- Filter rows based on date conditions.

```
# Between two dates (inclusive)
df[df['Date'].between('2024-01-01', '2024-01-31')]

# Before or after a date
df[df['Date'] < '2024-01-01']
df[df['Date'] >= '2024-01-01']
```

8. Date Differences

- Find difference between dates.

```
df['Days_Diff'] = (df['End_Date'] - df['Start_Date']).dt.days

df['Hours_Diff'] = (df['End_Date'] -
                    df['Start_Date']).dt.total_seconds() / 3600
```

9. Working with Periods

- Convert to period and extract period info.

```
df['Month_Period'] = df['Date'].dt.to_period('M')
df['Quarter_Period'] = df['Date'].dt.to_period('Q')
```

10. Resample Basics (Time Series)

- Resample time series data to different frequency.

Frequency	Meaning
D	Daily
W	Weekly
M	Month End
Q	Quarter End
Y	Year End

```
ts = df.set_index('Date').sort_index()
ts_daily   = ts.resample('D').sum()     # Daily
ts_weekly  = ts.resample('W').sum()     # Weekly
ts_monthly = ts.resample('M').mean()    # Monthly
```

11. Complete Example

- Extract month and filter data for March 2024.

```
df['Month'] = df['Date'].dt.month
march_data = df[df['Date'].dt.month == 3]
march_2024 = df(df['Date'] >= '2024-03-01') &
             (df['Date'] <= '2024-03-31')]
monthly_sales = df.set_index('Date').resample('M')['Sales'].sum()
```

PANDAS WITH PYTHON HANDBOOK

PAGE 8 : READING & WRITING DATA

1. Reading CSV Files

- Read data from a CSV (Comma Separated Values) file.

```
df = pd.read_csv('data.csv')                    # Basic usage
df = pd.read_csv('data.csv', header=0)          # First row as header
df = pd.read_csv('data.csv', usecols=['A','C'])
df = pd.read_csv('data.csv', nrows=100)
df = pd.read_csv('data.csv', na_values=['NA','-'])
df = pd.read_csv('data.csv', encoding='utf-8')
```

2. Writing CSV Files

- Write DataFrame to a CSV file.

```
df.to_csv('output.csv', index=False)      # Without index
df.to_csv('output.csv', index=True)       # With index
df.to_csv('output.csv', header=True)      # With header
df.to_csv('output.csv', sep=',')          # Custom separator
```

3. Reading Excel Files

- Read data from an Excel file.

```
df = pd.read_excel('data.xlsx')                  # First sheet
df = pd.read_excel('data.xlsx', sheet_name='Sheet2')
df = pd.read_excel('data.xlsx', usecols='A:D')
df = pd.read_excel('data.xlsx', engine='openpyxl')
df = pd.read_excel('data.xlsx', skiprows=2)
```

4. Writing Excel Files

- Write DataFrame to an Excel file.

```
df.to_excel('output.xlsx', index=False)      # Without index
df.to_excel('output.xlsx', sheet_name='Data')
df.to_excel('output.xlsx', index=False, header=True)
df.to_excel('output.xlsx', startrow=1, startcol=0)
df.to_excel('output.xlsx', engine='openpyxl')
```

5. Reading JSON Files

- Read data from a JSON file.

```
df = pd.read_json('data.json')                  # Default orient
df = pd.read_json('data.json', orient='records')
df = pd.read_json('data.json', lines=True)      # JSON Lines
df = pd.read_json('data.json', typ='frame')
```

6. Writing JSON Files

- Write DataFrame to a JSON file.

```
df.to_json('output.json')                  # Default orient='columns'
df.to_json('output.json', orient='records')
df.to_json('output.json', orient='split')
df.to_json('output.json', lines=True)      # JSON Lines
df.to_json('output.json', indent=4)
```

7. Important Parameters (read_csv)

Parameter	Description	Example
sep	Delimiter	sep=','
header	Row number to use as header	header=0
index_col	Column to use as row labels	index_col='ID'
usecols	Columns to read	usecols=['A','B']
na_values	Values to treat as NaN	na_values=['NA','-']
skiprows	Number of rows to skip	skiprows=2
nrows	Number of rows to read	nrows=100
dtype	Data type for columns	dtype={'A': str}
encoding	File encoding	encoding='utf-8'

8. Important Parameters (to_csv / to_excel)

Parameter	Description	Example
index	Write row index (True/False)	index=False
header	Write column names (True/False)	header=True
sep	Field delimiter (CSV only)	sep=';'
encoding	File encoding	encoding='utf-8'
sheet_name	Sheet name (Excel only)	sheet_name='Data'
startrow	Start writing at row	startrow=1
startcol	Start writing at column	startcol=0

9. Other Supported Formats

- Pandas supports reading/writing many other formats.

Format	Functions
HTML	read_html(), to_html()
SQL	read_sql(), to_sql()
Parquet	read_parquet(), to_parquet()
Pickle	read_pickle(), to_pickle()
HDF5	read_hdf(), to_hdf()
Clipboard	read_clipboard(), to_clipboard()

10. Tips

- Always check the file path and file name.
- Use raw string (r'path\to\file.csv') for Windows paths.
- Use try_except while reading files that may not exist.
- Large files? Use chunksize in read_csv() to read in parts.

```
for chunk in pd.read_csv('big_file.csv', chunksize=10000):
    process(chunk)
```

PANDAS WITH PYTHON HANDBOOK

PAGE 9 : HANDLING MISSING DATA

1. Introduction

- Real-world data often contains missing values.
- In Pandas, missing values are represented as NaN (Not a Number).

Common Missing Value Indicators

None, NaN, np.nan, NaT, blank cells, null, NULL, N/A, - , etc.

2. Detecting Missing Values

- Use isnull() / isna() to detect missing values.

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, None, 4],        # None → NaN
    'B': [np.nan, 5, 6, None]    # np.nan → NaN
})
df.isnull()       # Returns DataFrame of True/False
df.isna().sum()   # Count missing values per column
```

3. Dropping Missing Values

- Remove rows or columns with missing values.

```
df.dropna()                  # Drop rows with any NaN
df.dropna(how='all')         # Drop rows where all values are NaN
df.dropna(subset=['A'])      # Drop rows with NaN in column A
df.dropna(axis=1)            # Drop columns with any NaN
```

4. Filling Missing Values

- Replace NaN with a specific value or method.

```
df.fillna(0)                          # Fill NaN with 0
df.fillna('Unknown')                  # Fill NaN with string
df.fillna(df.mean(numeric_only=True)) # Fill with column mean
df.fillna(method='ffill')             # Forward fill
df.fillna(method='bfill')             # Backward fill
```

5. Interpolation

- Estimate missing values using interpolation.

```
df.interpolate()                # Linear interpolation
df.interpolate(method='time')   # Time-based interpolation
df.interpolate(limit=2)         # Limit consecutive NaNs filled
```

6. Replace Specific Values

- Replace specific placeholders with NaN or other values.

```
df.replace('N/A', np.nan, inplace=True)
df.replace(['-', 'NA', ''], np.nan, inplace=True)
df.replace(0, np.nan)    # Replace 0 with NaN
```

7. Working with NaT (Missing Dates)

- NaT is used for missing datetime values.

```
dates = pd.to_datetime(['2024-01-01', None, '2024-01-03'])
ser = pd.Series(dates)
ser.isna()     # Detect NaT
ser.fillna(pd.Timestamp('2024-01-02'))    # Fill NaT
```

8. Drop or Keep Based on Threshold

- Drop rows/columns if missing values exceed a threshold.

```
df.dropna(thresh=2)     # Keep rows with at least 2 non-NaN
df.dropna(axis=1, thresh=3) # Keep columns with at least
                            # 3 non-NaN
```

9. Visualizing Missing Data

- Simple way to see missing data distribution.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.isnull(), cbar=False, cmap='YlGnBu')
plt.title('Missing Data Heatmap')
plt.show()
```

Darker color = more missing values

Heatmap Color Guide

Color	Meaning
Light	Few or no missing values
Medium	Moderate missing values
Dark	High missing values

10. Summary Table

Method	Purpose	Example
isnull() / isna()	Detect missing values	df.isnull().sum()
dropna()	Remove missing data	df.dropna(), df.dropna(subset=['A'])
fillna()	Fill missing data	df.fillna(0), df.fillna(method='ffill')
interpolate()	Estimate missing values	df.interpolate(method='linear')
replace()	Replace placeholders	df.replace('N/A', np.nan)
thresh	Keep based on minimum non-NaN	df.dropna(thresh=3)
NaT handling	Missing datetime values	ser.fillna(pd.Timestamp('2024-01-01'))

PANDAS WITH PYTHON HANDBOOK

PAGE 10 : BEST PRACTICES & CHEAT SHEET

1. Best Practices

- Always inspect your data first using `head()`, `info()`, and `describe()`.
- Use meaningful column names.
- Handle missing data early in the analysis.
- Use vectorized operations (avoid loops).
- Chain methods using method chaining for clean code.
- Use `.loc[]` for label-based indexing (safer).
- Save intermediate results if the pipeline is long.
- Keep data types optimized (use category, int8, etc.).
- Document your steps and assumptions.

2. Performance Tips

- Use appropriate data types to save memory.
- Use `category` dtype for text columns with limited unique values.
- Use `.astype()` to downcast numeric types.
- Read only necessary columns using `usecols`.
- Process data in chunks for large files.
- Avoid `.apply()` with Python functions; use vectorized methods.
- Use `.query()` instead of boolean indexing when possible.
- Clear unused DataFrames to free memory.

3. Typical Data Analysis Workflow

1. Collect Data	CSV / Excel / SQL / API
2. Load Data	`pd.read_*()`
3. Explore Data	`head()`, `info()`, `describe()`
4. Clean Data	Handle missing, duplicates, types
5. Analyze Data	Filter, group, aggregate, visualize
6. Export Results	`to_csv()`, `to_excel()`, `to_json()`

4. Common Gotchas

- `SettingWithCopyWarning` occurs when modifying a slice. Use `.loc[]` to avoid it.
- `NaN != NaN` → use `pd.isna()`/`pd.notna()` for checks.
- Merging on columns with different dtypes may fail.
- Date parsing can be slow → specify format if known.
- `.apply()` is slow; prefer vectorized operations.
- Duplicated column names can cause unexpected results.
- Large DataFrames can crash memory → optimize early.

5. Pandas Cheat Sheet (Quick Reference)

Task	Method / Function	Example
Read CSV	`pd.read_csv()`	`pd.read_csv('data.csv')`
Read Excel	`pd.read_excel()`	`pd.read_excel('data.xlsx', sheet_name='Sheet1')`
View Top Rows	`head(n)`	`df.head(5)`
View Bottom Rows	`tail(n)`	`df.tail(5)`
Info Summary	`info()`	`df.info()`
Summary Statistics	`describe()`	`df.describe()`
Select Column	`df['col']`	`df['Name']`
Select Multiple Columns	`df[['A','B']]`	`df[['A','B']]`
Filter Rows	`df[df['A'] > 10]`	`df[df['Age'] > 18]`
Sort Values	`sort_values()`	`df.sort_values('Salary', ascending=False)`
Group By	`groupby()`	`df.groupby('Dept')['Salary'].sum()`
Merge / Join	`merge()`	`df.merge(df1, df2, on='ID', how='inner')`
Handle Missing	`fillna()` / `dropna()`	`df.fillna(0)` / `df.dropna()`
Create New Column	`df['New'] = ...`	`df['Total'] = df['Qty'] * df['Price']`
Drop Column	`drop(columns=...)`	`df.drop(columns=['A','B'])`
Export CSV	`to_csv()`	`df.to_csv('output.csv', index=False)`
Export Excel	`to_excel()`	`df.to_excel('output.xlsx', index=False)`

6. Final Checklist Before Analysis

- ☐ Data loaded successfully
- ☐ Column names are meaningful
- ☐ Data types are correct and optimized
- ☐ Missing values handled
- ☐ Duplicates checked & treated
- ☐ Outliers identified (if required)
- ☐ Data sorted / indexed properly
- ☐ Ready for analysis / visualization